



Table of Contents

Libraries 2

 CTypes 2

 CMath 2

 CStdlib and String 3

 STL Pair 3

 STL Tuple..... 3

 STL Vector..... 4

Data Structures & Algorithms 4

 STL Stack & Queue (No Iterators) 4

 STL Priority Queue (No Iterators)..... 4

 STL Map & Unordered Map 5

 STL Set & Unordered Set..... 5

 STL Algorithm & Numeric 5

 STL String Manipulation & String Stream 6

 STL Bit Manipulation & Bitset 7

 Union Find / Disjoint Set 7

 Graph 8

 Tree 12

Dynamic Programming 14

 Fibonacci Sequence 14

 0/1 Knapsack 14

 Unbounded Knapsack 15

 Longest Increasing Subsequence 15

 Subset Sum Problem 15

 String Matching 15

 Matrix Chain Multiplication 15

 Binomial Coefficient 15

 Maximum Sum 15

 Coin Change 15

 Number of Ways to Make Change 15

 Traveling Salesman Problem 16

.....

Number Theory 16

 GCD & LCM 16

 Modular Arithmetic 16

 Factorial 17

 Prime 17

 Divisor Count and Sum 18

Other Maths 18

 Matrices 18

 Arithmetic / Geometric Progressions..... 20

 Geometry 20

 Trigonometry 22

 Geometric Transformations with Matrix 22

Range Query 23

Random 24

Statistics 25

Libraries

Imports all libraries from `#include <bits/stdc++.h>`

```
#include <bits/stdc++.h>
using namespace std;

inline void fastIO() { // Cannot use with scanf() or printf()
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}

string int128ToString(i128 x){
    if (x == 0) return "0"; bool neg = x < 0; if (neg) x =- x; string s;
    while (x){ s.push_back(char('0'+ x % 10)); x /= 10; }
    if (neg) s.push_back('-'); reverse(s.begin(), s.end());
    return s;
}
```

```
typedef long long ll;          typedef __int128_t int128;  typedef vector<int> vi;
typedef vector<vi> vvi;       typedef pair<int, int> ii;   typedef vector<ii> vii;
typedef vector<vii> vvii;     typedef vector<bool> vb;    typedef tuple<int, int, int> iii;
const int INF = 0x7FFFFFFF, NEG_INF = -0x80000000;
```

Ctypes

Code	Type	Return type	Description
isalnum(int char)	function	bool	returns true if character is alphanumeric
isalpha(int char)	function	bool	returns true if character is alphabetic
isdigit(int char)	function	bool	returns true if character is digit from 1 to 9
isupper(int char)	function	bool	returns true if character is uppercase letter
islower(int char)	function	bool	returns true if character is lowercase letter
isxdigit(int char)	function	bool	returns true if character is hexadecimal digit
tolower(int char)	function	int	convert uppercase letter to lowercase
toupper(int char)	function	int	convert lowercase letter to uppercase

CMath (One radian is equivalent to $180/\text{PI}$ degrees where PI is 3.141592653... or use `M_PI` in the actual library)

Code	Type	Return type	Description
cos(double x)	function	double	returns the cosine of an angle of x radians
sin(double x)	function	double	returns the sine of an angle of x radians
tan(double x)	function	double	returns the tan of an angle of x radians
acos(double x)	function	double	returns the principal value of the arc cosine of x, expressed in radians (arc cosine is computed, in the interval $[-1,1]$ If the argument is out of this interval, a domain error occurs)
asin(double x)	function	double	returns the principal value of the arc sine of x, expressed in radians (arc sine is computed, in the interval $[-1,1]$ If the argument is out of this interval, a domain error occurs)
atan2(double x, double y)	function	double	returns the principal value of the arc tangent of y/x, expressed in radians where y representing the proportion of the y-coordinate and x representing the proportion of the x-coordinate (if both arguments passed are zero, a domain error occurs) Principal arc tangent of y/x, in the interval $[-\pi, \pi]$ radians
exp(double x)	function	double	returns the base-e exponential function of x
log(double x)	function	double	returns the base-e logarithm of x
log10(double x)	function	double	returns the base-10 logarithm of x
modf(double x, double* intpart)	function	double	breaks x into an integral and a fractional part where x will be rewrite with the integral



<code>exp2(double x)</code>	function	double	returns the base-2 exponential function of x
<code>exp2l(long double x)</code>	function	long double	
<code>pow(double base, double exponent)</code>	function	double	returns base raised to the power exponent
<code>sqrt(double x)</code>	function	double	returns the square root of x.
<code>cbrt(double x)</code>	function	double	returns the cubic root of x.
<code>cbrtl(long double x)</code>	function	long double	
<code>hypot(double x, double y)</code>	function	double	returns the hypotenuse of a right-angled triangle whose legs are x and y (the square root of $(x^2 + y^2)$)
<code>hypotl(long double x, long double y)</code>	function	long double	
<code>ceil(double x)</code>	function	double	rounds x upward, returning the smallest integral value that is not less than x
<code>floor(double x)</code>	function	double	rounds x downward, returning the largest integral value that is not greater than x
<code>fmod(double numer, double denom)</code>	function	double	returns the floating-point remainder of numer/denom (rounded towards zero)
<code>trunc(double x)</code>	function	double	rounds x toward zero, returning the nearest integral value that is not larger in magnitude than x
<code>truncl(long double x)</code>	function	long double	
<code>round(double x)</code>	function	double	returns the integral value that is nearest to x, with halfway cases rounded away from zero
<code>roundl(long double x)</code>	function	long double	
<code>double abs(double x)</code>	function	double	returns the absolute value of x

CStdlib and String

Code	Type	Return type	Description
<code>atof(const char* s)</code>	function	double	convert string to double
<code>stod(const string& s, size_t* idx = 0, int base = 10)</code>	function	double	
<code>stold(const string& s, size_t* idx = 0, int base = 10)</code>	function	long double	convert string to long double
<code>atoi(const char* s)</code>	function	int	convert string to int
<code>stoi(const string& s, size_t* idx = 0, int base = 10)</code>	function	int	
<code>atol(const char* s)</code>	function	long int	convert string to long int
<code>stol(const string& s, size_t* idx = 0, int base = 10)</code>	function	long int	
<code>atoll(const char* s)</code>	function	long long int	convert string to long long int
<code>stoll(const string& s, size_t* idx = 0, int base = 10)</code>	function	long long int	convert string to long long int
<code>to_string(T val)</code>	function	string	returns a string with the representation of val

STL Default Values Of Constructor : Default value is 0 if type is int, 0.0 if float, false if bool, "" if string

STL Pair

Code	Type	Return type	Description
<code>pair<T, T> p</code>	pair	-	constructs a new pair
<code>make_pair(T a, T b)</code>	function	pair	creates a pair object of type, determined by the argument types (a, b)
<code>{a, b}</code>	function	pair	
<code>first</code>	attribute	<code>typeof<first></code>	returns the first element of a pair
<code>second</code>	attribute	<code>typeof<second></code>	returns the second element of a pair

STL Tuple

Code	Type	Return type	Description
<code>tuple<T, T, type, ...> t</code>	tuple	-	constructs a new tuple
<code>make_tuple(a, b, c, ...)</code>	function	tuple	creates a tuple object of type, determined by the argument types (a, b, c, ...)
<code>{a, b, c, ...}</code>	function	tuple	
<code>get<int i>(t)</code>	function	<code>typeof<value></code>	accesses the ith element of a tuple (either getter or setter)



<code>tie(a, b, c, ...) = t;</code>	function	-	unpacks a tuple object into variables
-------------------------------------	----------	---	---------------------------------------

STL Vector

Code	Type	Return type	Description
<code>vector<T> v</code>	vector	-	constructs a new empty vector
<code>vector<type> v(int n, T k)</code>	vector	-	constructs a new vector with size of n and every element is k
<code>vector<type> v(vector<T> v1)</code>	vector	-	constructs new vector that copies vector v1
<code>v.size()</code>	method	int	returns the size of the vector
<code>v.empty()</code>	method	bool	returns true if the vector is empty
<code>v.front()</code>	method	<code>typeof<value></code>	returns the value of first element in the vector
<code>v.back()</code>	method	<code>typeof<value></code>	returns the value of last element in the vector
<code>v.begin()</code>	method	iterator	returns iterator pointing to beginning
<code>v.end()</code>	method	iterator	returns iterator pointing to end
<code>v.resize(int k)</code>	method	void	changes the size of the vector to k
<code>v.clear()</code>	method	void	removes every element in the vector
<code>v1.swap(vector<T> v2)</code>	method	void	swaps between two vectors
<code>for(auto &i : vector<T> v) cin >> i;</code>	-	-	inputs each elements of the vector using auto
<code>v.push_back(T x)</code>	method	void	appends a new element at the back of the vector
<code>v.insert(iterator i, T x)</code>	method	void	inserts a new element at the i position which i has to be v.begin() + k where k is the index
<code>v.pop_back()</code>	method	void	removes the last element of the vector
<code>v.erase(iterator i)</code>	method	void	removes the element at the i position which i has to be v.begin() + k where k is the index
<code>v.erase(iterator i, iterator j)</code>	method	void	removes elements from i position to j - 1 position which i, j has to be v.begin() + k, v.begin() + l respectively, where k, l are the indices and k <= l
<code>v.emplace(const_iterator position, Args&&... args)</code>	method	void	inserts a new element at the i position which i has to be v.begin() + k where k is the index whilst the rest arguments will be used to construct an object
<code>v.emplace_back(Args&&... args)</code>	method	void	inserts a new element at the last position and the arguments will be used to construct an object

Data Structures & Algorithms

STL Stack & Queue (No Iterators) : Stack uses LIFO, queue uses FIFO

Code	Type	Return type	Description
<code>stack<T> s</code>	stack	-	constructs a new stack
<code>queue<T> q</code>	queue	-	constructs a new queue
<code>s.push(T x)</code>	method	void	appends a new element to the stack
<code>q.push(T x)</code>	method	void	appends a new element to the queue
<code>s.pop()</code>	method	void	removes the last element of the stack
<code>q.pop()</code>	method	void	removes the first element of the queue
<code>s.size()</code>	method	int	returns the size of the stack
<code>q.size()</code>	method	int	returns the size of the queue
<code>s.top()</code>	method	T	returns the top element of the stack
<code>q.front()</code>	method	T	returns the first element of the queue
<code>q.back()</code>	method	T	returns the last element of the queue

STL Priority Queue (No Iterators)

Code	Type	Return type	Description
<code>priority_queue<T> pq</code>	priority_queue	-	constructs a new priority queue



<code>priority_queue<T, vector<T>, comp> pq</code>	priority_queue	-	constructs a new priority queue with a specific comparator (by default: <code>less<vector<T>>></code>)
<code>pq.push(T x)</code>	method	void	appends a new element to the priority queue whose elements will be sorted by priority (by default is based on descending order)
<code>pq.pop()</code>	method	void	removes the first element of the queue
<code>pq.empty()</code>	method	bool	returns true if the queue is empty
<code>pq.top()</code>	method	<code>typeof<value></code>	returns the first element of the queue
<code>pq.size()</code>	method	int	returns the size of the queue
<code>for (i = 0; i < pq.size(); i++) { cout << pq.top(); pq.pop(); }</code>	-	-	prints and remove each element of the queue (since the queue has no iterator , this is the only way to print all elements)

STL Map & Unordered Map

Code	Type	Return type	Description
<code>map<T, T> m</code>	map	-	constructs a new map whose entries always be sorted by the keys based on ascending order
<code>unordered_map<T, T> um</code>	unordered_map	-	constructs a new unordered map whose entries not always be sorted
<code>map<T, T> m(iterator i, iterator j)</code>	map	-	constructs a new map from elements from i position to j - 1 position of a vector of pairs, which i, j has to be v.begin() + k, v.begin() + l respectively, where k, l are the indices and k <= l (j also can be <code>v.end()</code>)
<code>map<T, T, comp> m</code>	map	-	constructs a new map with a specific key comparator (by default: <code>less<typeof<key>>></code>)
<code>m[x] = y</code>	operator	-	appends a new entry to the map or updates the value of the entry matching the key
<code>m.insert({T x, T y})</code>	method	void	
<code>m[x]</code>	operator	<code>typeof<value></code>	returns the value of the entry matching the key
<code>for(auto i : map<T, T> m) cout << i.first;</code>	-	-	prints all keys of the entries in the map using auto
<code>for(auto i : map<T, T> m) cout << i.second;</code>	-	-	prints all values of the entries in the map using auto
<code>m.erase(T x)</code>	method	void	removes the entry matching the key
<code>m.count(T x)</code>	method	int	returns the number of entries matching the key
<code>m.find(T x)</code>	method	iterator	returns the iterator of the entry matching the key and returns <code>m.end()</code> if there is no matching key

STL Set & Unordered Set

Code	Type	Return type	Description
<code>set<T> s</code>	set	-	constructs a new set whose elements always be sorted based on ascending order
<code>unordered_set<T> us</code>	unordered_set	-	constructs a new unordered set whose elements not always be sorted
<code>s.insert(T x)</code>	method	void	appends a new element to the set
<code>next(iterator si, int i)</code>	function	iterator	returns an iterator pointing to the element of the set at index i
<code>s.erase(T x)</code>	method	void	removes the element by value
<code>s.contains(T x)</code>	method	bool	returns true if the element is present in the set

STL Algorithm & Numeric

Code	Type	Return type	Description
<code>iota(iterator first, iterator last, int i)</code>	function	void	fills a range of elements with increasing values starting from the initial value
<code>min_element(iterator first, iterator last)</code>	function	iterator	returns an iterator pointing to the minimum element in the



			range
<code>max_element(iterator first, iterator last)</code>	function	iterator	returns an iterator pointing to the maximum element in the range
<code>sort(iterator first, iterator last)</code>	function	void	sorts all elements in the range based on ascending order
<code>sort(iterator first, iterator last, func<bool> comp)</code>	function	void	sorts all elements in the range with a specific comparator
<code>lower_bound(iterator first, iterator last, T i)</code>	function	iterator	returns an iterator pointing to the first element that is greater than or equal to the specified value
<code>upper_bound(iterator first, iterator last, T i)</code>	function	iterator	returns an iterator pointing to the first element that is greater than the specified value
<code>accumulate(iterator first, iterator last, T i)</code>	function	double	returns the sum of all the values lying in the range and the initial value
<code>accumulate(iterator first, iterator last, T i, function op)</code>	function	double	returns the sum of all the values lying in the range and the initial value with a specific operation
<code>reverse(iterator first, iterator last)</code>	function	void	reverses the order of elements in the given range of elements
<code>next_permutation(iterator first, iterator last)</code>	function	void	rearranges the elements of the given range to the lexicographical larger permutation if it exists.
<code>prev_permutation(iterator first, iterator last)</code>	function	void	rearranges the elements of the given range to the lexicographical smaller permutation if it exists.
<code>shuffle(iterator first, iterator last, URNG g)</code>	function	void	Rearranges the elements of the given range randomly, using g as Uniform Random Number Generator (URNG)
<code>search(iterator first, iterator last, iterator s_first, iterator s_last)</code>	function	iterator	returns the iterator of the first occurrence of the sequence of elements [s_first, s_last) in the range [first, last)
<code>unique(iterator first, iterator last)</code>	function	void	removes duplicates of any element present consecutively in a the given range
<code>rotate(iterator first, iterator mid, iterator last)</code>	function	void	rotates the elements of a range in left (mid = first + n; n is the index) or right (mid = first + last - n) direction such that mid becomes the new first element of the range.
<code>partition(iterator first, iterator last, func<bool> pred)</code>	function	void	reorders elements within the given range based on a given condition or predicate whilst not preserving the relative order of elements within each partition
<code>stable_partition(iterator first, iterator last, func<bool> p)</code>	function	void	reorders elements within the given range based on a given condition or predicate p whilst preserving the relative order of elements within each partition
<code>partial_sort(iterator first, iterator mid, iterator last)</code>	function	void	rearranges the elements of the given range that only the elements before middle are sorted in ascending order
<code>is_sorted(iterator first, iterator last)</code>	function	bool	returns if the elements of the given range is sorted
<code>binary_search(iterator first, iterator last, T value)</code>	function	bool	returns if there is a value in the elements of the given range
<code>lexicographical_compare(iterator first1, iterator last1, iterator first2, iterator last2)</code>	function	bool	checks if the first range [first1, last1) is lexicographically less than the second range [first2, last2)
<code>find(iterator first, iterator last, T value)</code>	function	iterator	returns the first element in a given range that is equal to a specified value
<code>find_if(iterator first, iterator last, func<bool> p)</code>	function	iterator	returns the first element in a given range for which a provided unary predicate p returns true
<code>find_if_not(iterator first, iterator last, func<bool> q)</code>	function	iterator	returns the first element in a given range for which a provided unary predicate q returns false

STL String Manipulation & String Stream

Code	Type	Return type	Description
<code>s.length()</code>	method	int	returns the size of the string
<code>s.substr(int p, int l)</code>	method	string	returns the substring of the string starting from p to p + l - 1



<code>s.erase(int p, int l)</code>	method	string	removes the part of the string starting from p to p + l - 1
<code>s1 += s2</code>	operator	-	concatenates a string to another string
<code>s1.replace(int pos, int n, string s2)</code>	method	void	replaces the n characters of the string s1 starting from the index pos with a string s2
<code>s1.replace(iterator i, iterator j, string s2)</code>	method	void	replaces the characters of the string s1 starting from i position to j - 1 position with a string s2, which i, j has to be s1.begin() + k, s1.begin() + l respectively , where k, l are the indices and k <= l (j also can be s1.end())
<code>s1.find(string s2)</code>	method	int	returns the index of the first occurrence of a string s2 in the string s1 and returns -1 if there is no occurrence
<code>s1.rfind(string s2)</code>	method	int	returns the index of the last occurrence of a string s2 in the string s1 and returns -1 if there is no occurrence
<code>getline(cin, string s, string delimiter)</code>	function	void	inputs the entire line of the string until the specified delimiter is found (by default is the newline character)
<code>stringstream ss(string s1)</code>	stringstream	-	construct a new stringstream with the string
<code>while (ss >> s2) cout << s2</code>	-	-	prints all strings divided by the stringstream based on whitespace

STL Bit Manipulation & Bitset

Code	Type	Return type	Description
<code>int a ^ int b</code>	operator	int	returns an integer from XOR (Exclusive OR) operation
<code>~ int a</code>	operator	int	returns an complementary integer which equals to - (a + 1)
<code>int a << int b</code>	operator	int	returns an integer from left-shift operation which equals to a * 2^b
<code>int a >> int b</code>	operator	int	returns an integer from right-shift operation which equals to a / 2^b
<code>bitset b(int a)</code>	bitset	-	construct a new bitset with the integer
<code>b.set(int i)</code>	method	void	set a value at the i position of the bitset to 1
<code>b.reset(int i)</code>	method	void	set a value at the i position of the bitset to 0
<code>b.flip(int i)</code>	method	void	set a value at the i position of the bitset to its complement
<code>b.count()</code>	method	int	returns the number of all 1 values in the bitset

Union Find / Disjoint Set

```
vector<int> par = vector<int>(n + 1);    // par = parent
iota(par.begin(), par.end(), 0);

vector<int> rank = vector<int>(n + 1, 0), size = vector<int>(n + 1, 1);

int findUPar(int node) { return par[node] == node ? node : (par[node] = findUPar(par[node])); }

void unionByRank(int u, int v) {
    int ulp_u = findUPar(u), ulp_v = findUPar(v);
    if (ulp_u == ulp_v) return;
    if (rank[ulp_u] < rank[ulp_v]) par[ulp_u] = ulp_v;
    else if (rank[ulp_v] < rank[ulp_u]) par[ulp_v] = ulp_u;
    else { par[ulp_v] = ulp_u; rank[ulp_u]++; }
}

void unionBySize(int u, int v) {
    int ulp_u = findUPar(u), ulp_v = findUPar(v);
    if (ulp_u == ulp_v) return;
    if (size[ulp_u] < size[ulp_v]) { par[ulp_u] = ulp_v; size[ulp_v] += size[ulp_u]; }
    else { par[ulp_v] = ulp_u; size[ulp_u] += size[ulp_v]; }
}

bool sameSet(int u, int v) { return findUPar(u) == findUPar(v); }
```

Graph

Note: if uses map you have to check if the key exists first by `if (graph.find(s) != graph.end()) { ... }`

```
vector<vector<int>> graph = vector<vector<int>>(n, vector<int>(n, 0)); // matrix
map<int, vector<int>> graph; // list (automated)

void addEdge(int u, int v) {
    graph[u][v] = 1;
    graph[v][u] = 1; // if undirected
}

void addEdge(int u, int v) {
    graph[u].push_back(v);
    graph[v].push_back(u); // if undirected
}

void dfs(vector<vector<int>>& graph, vector<bool>& visit, int node, vector<int>& result) {
    visit[node] = true;
    result.push_back(node);
    for (int neigh : graph[node]) if (!visit[neigh]) dfs(graph, visit, neigh, result);
}

vector<int> bfs(vector<vector<int>>& graph, int start = 0) {
    int n = graph.size();
    vector<int> result;
    vector<bool> visit(n, false);
    queue<int> q;
    q.push(start);
    visit[start] = true;

    while (!q.empty()) {
        int current = q.front();
        q.pop();
        result.push_back(current);
        for (int neighbor : graph[current])
            if (!visit[neighbor]) { visit[neighbor] = true; q.push(neighbor); }
    }

    return result;
}

void findAllPaths(int current, int target, vector<vector<int>>& graph,
                 vector<int>& path, vector<vector<int>>& allPaths) {
    path.push_back(current);
    if (current == target) allPaths.push_back(path);
    else for (int neigh : graph[current]) findAllPaths(neigh, target, graph, path, allPaths);
    path.pop_back();
}

struct TJSCC { // Tarjan's algorithm: solving for strongly connected component
    vector<vector<int>> graph, sccs;
    vector<int> ids, low;
    vector<bool> onStack;
    stack<int> stk;
    int idC, n; // idC = idCounter

    TJSCC (vector<vector<int>>& adjList) : graph(adjList), n(adjList.size()), idC(0) {
        ids.assign(n, -1);
        low.assign(n, -1);
        onStack.assign(n, false);
    }

    vector<vector<int>> findSCCs() {
        for (int i = 0; i < n; i++) { if (ids[i] == -1) dfs(i); }
        return sccs;
    }

    void dfs(int at) {
        stk.push(at);
        onStack[at] = true;
        ids[at] = low[at] = idC++;
        for (int to : graph[at])
            if (ids[to] == -1) {
```



```

        dfs(to);
        low[at] = min(low[at], low[to]);
    } else if (onStack[to]) low[at] = min(low[at], ids[to]);

    if (ids[at] == low[at]) {
        vector<int> scc;
        int node;
        do {
            node = stk.top();
            stk.pop();
            onStack[node] = false;
            scc.push_back(node);
        } while (node != at);

        sccs.push_back(scc);
    }
}

};

void floydWarshall(vector<vector<int>>& d) {    // d = dist
    int V = d.size();
    for (int k = 0; k < V; k++) for (int i = 0; i < V; i++) for (int j = 0; j < V; j++)
        if (d[i][k] != INF && d[k][j] != INF) d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}

// edge: (u, v, w) format, d = dist
vector<int> bellmanFord(int V, vector<tuple<int, int, int>>& edges, int src) {
    vector<int> d(V, INF);
    d[src] = 0;
    for (int i = 0; i < V; i++) for (auto& e : edges) {
        int u, v, wt;
        tie(u, v, wt) = e;
        if (d[u] != INF && d[u] + wt < d[v]) {
            if (i == V - 1) return {-1};
            d[v] = d[u] + wt;
        }
    }

    return d;
}

vector<int> topologicalSort(vector<vector<int>>& graph) {
    int n = graph.size();
    vector<int> in(n, 0), result;    // in = inDegree
    for (int u = 0; u < n; u++) for (int v : graph[u]) in[v]++;

    queue<int> q;
    for (int i = 0; i < n; i++) if (in[i] == 0) q.push(i);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        result.push_back(u);
        for (int v : graph[u]) {
            in[v]--;
            if (in[v] == 0) q.push(v);
        }
    }
    if (result.size() != n) return {};
    return result;
}

vector<int> articulationPoints(int V, vector<vector<int>>& graph) {
    vector<int> result;
    for (int i = 0; i < V; i++) {
        vector<bool> visit(V, false);
        visit[i] = true;
        int comps = 0;    // comps = components
        function<void(int)> dfs;
        dfs = [&](int node) {
            visit[node] = true;
            for (int neigh : graph[node]) if (!visit[neigh]) dfs(neigh);
        };
    }
}

```

```

        for (int neigh : graph[i]) {
            if (comps > 1) break;
            if (!visit[neigh]) { dfs(neigh); comps++; }
        }
        if (comps > 1) result.push_back(i);
    }
    return result.empty() ? vector<int>{-1} : result;
}

struct BF {    // BF = Bridge Finder
    vector<vector<int>>> graph;
    vector<pair<int,int>>> bridges;
    vector<int> disc, low;
    vector<bool> visit;
    int timer, n;

    BF(vector<vector<int>>& adjList) : graph(adjList), n(adjList.size()), timer(0) {
        disc.assign(n, -1);
        low.assign(n, -1);
        visit.assign(n, false);
    }

    vector<pair<int,int>>> findBridges() {
        for (int i = 0; i < n; i++) if (!visit[i]) dfs(i, -1);
        return bridges;
    }

    void dfs(int u, int parent) {
        visit[u] = true;
        disc[u] = low[u] = timer++;
        for (int v : graph[u]) {
            if (v == parent) continue;
            if (visit[v]) low[u] = min(low[u], disc[v]);
            else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] > disc[u]) bridges.push_back({u, v});
            }
        }
    }
};

// Prim's MST using Priority Queue : O(E)
vector<pair<int, int>>> primsMST(vector<vector<pair<int, int>>>& graph, int& totalCost) {
    int n = graph.size(), totalCost = 0;
    vector<bool> inMST(n, false);
    vector<pair<int, int>>> mstEdges;
    priority_queue<pair<int, pair<int, int>>, vector<pair<int, pair<int, int>>>,
        greater<pair<int, pair<int, int>>>> pq;
    inMST[0] = true;
    for (auto& e : graph[0]) {
        int neigh = e.first, w = e.second;
        pq.push({w, {0, neigh}});
    }

    while (!pq.empty() && mstEdges.size() < n - 1) {
        auto current = pq.top();
        pq.pop();

        int w = current.first, u = current.second.first, v = current.second.second;
        if (inMST[v]) continue;        // Skip if already in MST

        inMST[v] = true;
        mstEdges.push_back({u, v});
        totalCost += w;
        for (auto& e : graph[v]) {    // e = edge
            int neigh = e.first;
            if (!inMST[neigh]) pq.push({e.second, {v, neigh}});
        }
    }
    return mstEdges;
}

```

```

void primsMST(vector<vector<pair<int, int>>>& graph, int& totalCost) { // O(V)
    int n = graph.size(), totalCost = 0;
    vector<bool> inMST(n, false);
    vector<int> key(n, INF);
    key[0] = 0;

    for (int count = 0; count < n; count++) {
        int u = -1;
        for (int v = 0; v < n; v++) if (!inMST[v] && (u == -1 || key[v] < key[u])) u = v;
        inMST[u] = true;
        totalCost += key[u];
        for (auto& e : graph[u]) {
            int v = e.first, w = e.second;
            if (!inMST[v] && w < key[v]) key[v] = w;
        }
    }
}

void kruskalsMST(vector<vector<int>>& edges, int V, int& totalCost) {
    vector<int> parent = vector<int>(V + 1);
    iota(parent.begin(), parent.end(), 0);
    vector<int> rank = vector<int>(V + 1, 0), size = vector<int>(V + 1, 1);
    sort(edges.begin(), edges.end(), [](vector<int>& a, vector<int>& b) { return a[2] < b[2]; });
    int edgesUsed = 0, totalCost = 0;
    for (auto& e : edges) {
        int u = e[0], v = e[1], weight = e[2];
        if (!sameSet(u, v)) {
            unionByRank(u, v); // or use unionBySize(u, v)
            totalCost += weight;
            edgesUsed++;
            if (edgesUsed == V - 1) break;
        }
    }
}

struct BipartiteGraph { // Maximum Bipartite Matching (Hopcroft-Karp Algorithm)
    int num_u_vertices, num_v_vertices;
    vector<vector<int>>> adj;
    vector<int> match_u, match_v, dist;
    bool bfs() { // BFS to build the layered graph
        queue<int> q;
        dist.assign(num_u_vertices + 1, INF);
        for (int u = 1; u <= num_u_vertices; ++u) if (match_u[u] == 0) { dist[u] = 0; q.push(u); }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            if (dist[u] < dist[0])
                for (int v : adj[u]) {
                    int matched_u = match_v[v];
                    if (dist[matched_u] == INF) {
                        dist[matched_u] = dist[u] + 1; q.push(matched_u);
                    }
                }
        }
        return dist[0] != INF;
    }

    bool dfs(int u) { // DFS to find augmenting paths
        if (u != 0) {
            for (int v : adj[u]) {
                int matched_u = match_v[v];
                if (dist[matched_u] == dist[u] + 1) {
                    if (dfs(matched_u)) continue;
                    match_v[v] = u; match_u[u] = v; return true;
                }
            }
            dist[u] = INF;
            return false;
        }
        return true;
    }
}

```

```

BipartiteGraph(int u_size, int v_size) : num_u_vertices(u_size), num_v_vertices(v_size) {
    adj.resize(u_size + 1);
    match_u.assign(u_size + 1, NIL);
    match_v.assign(v_size + 1, NIL);
}

void add_edge(int u, int v) { adj[u].push_back(v); }

int max_matching() {
    int result = 0;
    while (bfs()) for (int u = 1; u <= num_u_vertices; ++u)
        if (match_u[u] == NIL && dfs(u)) result++;
    return result;
}
};

```

Tree

```

struct Node {
    int data;
    Node *left, *right;
    Node(int x) { data = x; left = right = nullptr; }
};

void inorder(Node* root, vector<int>& result) {
    if (root == nullptr) return;
    inorder(root->left, result);
    result.push_back(root->data);
    inorder(root->right, result);
}

void preorder(Node* root, vector<int>& result) {
    if (root == nullptr) return;
    result.push_back(root->data);
    preorder(root->left, result);
    preorder(root->right, result);
}

void postorder(Node* root, vector<int>& result) {
    if (root == nullptr) return;
    postorder(root->left, result);
    postorder(root->right, result);
    result.push_back(root->data);
}

vector<vector<int>> levelOrder(Node* root) {
    vector<vector<int>> result;
    if (root == nullptr) return result;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        int size = q.size();
        vector<int> level;
        for (int i = 0; i < size; i++) {
            Node* node = q.front(); q.pop();
            level.push_back(node->data);
            if (node->left) q.push(node->left); if (node->right) q.push(node->right);
        }
        result.push_back(level);
    }
    return result;
}

int height(Node* root) {
    if (root == nullptr) return 0;
    return 1 + max(height(root->left), height(root->right));
}

int diameter(Node* root) {
    if (root == nullptr) return 0;
    int leftHeight = height(root->left), rightHeight = height(root->right),
        leftDiameter = diameter(root->left), rightDiameter = diameter(root->right);
    return max(leftHeight + rightHeight + 1, max(leftDiameter, rightDiameter));
}

```



```
bool isBalanced(Node* root) {
    if (root == nullptr) return true;
    int leftHeight = height(root->left), rightHeight = height(root->right);
    return abs(leftHeight - rightHeight) <= 1 && isBalanced(root->left) &&
        isBalanced(root->right);
}

bool search(Node* root, int target) {
    if (root == nullptr) return false;
    if (root->data == target) return true;
    return search(root->left, target) || search(root->right, target);
}

bool isValidBST(Node* root, int minVal = INT_MIN, int maxVal = INT_MAX) {
    if (root == nullptr) return true;
    if (root->data <= minVal || root->data >= maxVal) return false;
    return isValidBST(root->left, minVal, root->data) &&
        isValidBST(root->right, root->data, maxVal);
}

Node* insertBST(Node* root, int val) {
    if (root == nullptr) return new Node(val);
    if (val < root->data) root->left = insertBST(root->left, val);
    else if (val > root->data) root->right = insertBST(root->right, val);
    return root;
}

Node* findMin(Node* root) { while (root && root->left) root = root->left; return root; }

Node* deleteBST(Node* root, int val) {
    if (root == nullptr) return root;
    if (val < root->data) root->left = deleteBST(root->left, val);
    else if (val > root->data) root->right = deleteBST(root->right, val);
    else {
        if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        } else if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
        Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteBST(root->right, temp->data);
    }
    return root;
}

Node* buildFromInorderPreorder(vector<int>& preorder, vector<int>& inorder) {
    if (preorder.empty() || inorder.empty()) return nullptr;
    Node* root = new Node(preorder[0]);
    auto it = find(inorder.begin(), inorder.end(), preorder[0]);
    int idx = it - inorder.begin();
    vector<int> leftInorder(inorder.begin(), inorder.begin() + idx),
        rightInorder(inorder.begin() + idx + 1, inorder.end()),
        leftPreorder(preorder.begin() + 1, preorder.begin() + 1 + leftInorder.size()),
        rightPreorder(preorder.begin() + 1 + leftInorder.size(), preorder.end());
    root->left = buildFromInorderPreorder(leftPreorder, leftInorder);
    root->right = buildFromInorderPreorder(rightPreorder, rightInorder);
    return root;
}

bool hasPathSum(Node* root, int targetSum) {
    if (root == nullptr) return false;
    if (root->left == nullptr && root->right == nullptr) return root->data == targetSum;
    return hasPathSum(root->left, targetSum - root->data) ||
        hasPathSum(root->right, targetSum - root->data);
}
```



```
void findAllPaths(Node* root, int targetSum, vector<int>& path, vector<vector<int>>& result) {
    if (root == nullptr) return;
    path.push_back(root->data);
    if (root->left == nullptr && root->right == nullptr && root->data == targetSum)
        result.push_back(path);
    else { findAllPaths(root->left, targetSum - root->data, path, result);
          findAllPaths(root->right, targetSum - root->data, path, result);
    }
    path.pop_back();
}

Node* lowestCommonAncestor(Node* root, Node* p, Node* q) {
    if (root == nullptr || root == p || root == q) return root;
    Node* left = lowestCommonAncestor(root->left, p, q);
    Node* right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root;
    return left ? left : right;
}

int mxPathSum(Node *root, int &maxPathSum) {
    if (root == nullptr) return 0;
    int mxLeft = mxPathSum(root->left, maxPathSum), mxRight = mxPathSum(root->right, maxPathSum);
    if (root->left != nullptr && root->right != nullptr) {
        int maxSumPathViaNode = mxLeft + mxRight + root->data;
        maxPathSum = max(maxPathSum, maxSumPathViaNode);
        return root->data + max(mxLeft, mxRight);
    }
    return root->data + (root->left ? mxLeft : mxRight);
}

int findMaxSum(Node* root) {
    if (root == nullptr) return 0;
    int maxPathSum = INT_MIN;
    mxPathSum(root, maxPathSum);
    return maxPathSum;
}

struct BIT { // Fenwick Tree
    int n;
    vector<long long> t;
    BIT(int n) : n(n), t(n+1,0) {}
    void add(int i, long long v) { for(; i <= n; i += i & -i) t[i] += v; }
    long long sum(int i) { long long r = 0; for (; i > 0; i -= i & -i) r += t[i]; return r; }
    long long range(int l, int r) { return sum(r) - sum(l-1); }
};
```

Dynamic Programming (Recursion Memoization/Tabulation)

Fibonacci Sequence

Base Case	$F(0) = 0, F(1) = 1$
Recursive Case	$F(n) = F(n-1) + F(n-2)$
<pre>pair<long long, long long> fib_fast(long long n) { if (n == 0) return {0, 1}; auto pr = fib_fast(n >> 1); long long a = pr.first; // F_k long long b = pr.second; // F_{k+1} long long c = a * ((2 * b - a)); // F_{2k} long long d = a * a + b * b; // F_{2k+1} if ((n & 1) == 0) return {c, d}; else return {d, c + d}; } long long fib(long long n) { return fib_fast(n).first; }</pre>	

0/1 Knapsack

Base Case	$\text{knapsack}(0, w) = 0, \text{knapsack}(i, 0) = 0$
Recursive Case	<pre>knapsack(i, w) = max(knapsack(i-1, w), // don't take item i knapsack(i-1, w-weight[i]) + value[i] // take item i (if it fits))</pre>

**Unbounded Knapsack (Unlimited items of each type)**

Base Case	<code>knapsack(0) = 0</code>
Recursive Case	<code>knapsack(w) = max(knapsack(w - weight[i]) + value[i])</code> for all items <code>i</code> where <code>weight[i] <= w</code>

Longest Increasing Subsequence (LIS)

Base Case	<code>LIS[i] = 1</code> (each element forms a subsequence of length 1)
Recursive Case	<code>LIS[i] = max(LIS[j] + 1)</code> for all <code>j < i</code> where <code>arr[j] < arr[i]</code>

Subset Sum Problem

Base Case	<code>subsetSum(0, any_set) = True</code> (sum 0 is always possible with empty subset) <code>subsetSum(sum, empty_set) = False</code> (can't make positive sum with no elements)
Recursive Case	<code>subsetSum(sum, set) = subsetSum(sum, set - last)</code> OR <code>subsetSum(sum - last, set - last)</code>

String Matching (Edit Distance/Levenshtein Distance)

Base Case	<code>editDistance("", s2) = len(s2)</code> (insert all characters) <code>editDistance(s1, "") = len(s1)</code> (delete all characters)
Recursive Case	<pre>editDistance(s1, s2) = { if s1[0] == s2[0] editDistance(s1[1:], s2[1:]) if s1[0] != s2[0] return 1 + min(editDistance(s1[1:], s2), // delete editDistance(s1, s2[1:]), // insert editDistance(s1[1:], s2[1:])) // replace }</pre>

Matrix Chain Multiplication

Base Case	<code>MCM(i, i) = 0</code> (single matrix needs no multiplication)
Recursive Case	<code>MCM(i, j) = min(MCM(i, k) + MCM(k+1, j) + cost(i, k, j))</code> for all <code>k</code> in <code>[i, j-1]</code> <code>cost(i, k, j) = p[i-1] × p[k] × p[j]</code>

Binomial Coefficient (nCr)

Base Case	<code>C(n, 0) = 1</code> (one way to choose 0 items) <code>C(n, n) = 1</code> (one way to choose all items) <code>C(n, r) = 0</code> if <code>r > n</code>
Recursive Case	<code>C(n, r) = C(n-1, r-1) + C(n-1, r)</code> (Pascal's triangle)

Maximum Sum

Base Case	<code>maxSum[0] = arr[0]</code> (only first element) <code>maxSum[1] = max(arr[0], arr[1])</code> (better of first two)
Recursive Case	<code>maxSum[i] = max(maxSum[i-1], maxSum[i-2] + arr[i])</code>

Coin Change

Base Case	<code>coinChange(0) = 0</code> (0 coins for amount 0) <code>coinChange(amount < 0) = ∞</code> (impossible)
Recursive Case	<code>coinChange(amount) = min(coinChange(amount - coin) + 1)</code>

Number of Ways to Make Change

Base Case	<code>ways(0) = 1</code> (one way to make 0: use no coins) <code>ways(amount < 0) = 0</code> (no way to make negative amount)
Recursive Case	<code>ways(amount) = sum(ways(amount - coin))</code> for all valid coins

Traveling Salesman Problem

- $dp[mask][i]$ = minimum cost to visit all cities in mask and end at city i
- mask is a bitmask where bit j is 1 if city j has been visit

Base Case	$dp[1][0] = 0$ (start at city 0, only city 0 visited) $dp[mask][i] = INF$ for invalid states
Recursive Case	$dp[mask][i] = \min(dp[mask \wedge (1 \ll i)][j] + dist[j][i])$ for all j where bit j is set in mask $\wedge (1 \ll i)$ and $j \neq i$

Number Theory

GCD & LCM

```

long long gcd(long long a, long long b) { return b == 0 ? a : gcd(b, a % b); }

long long lcm(long long a, long long b) { return a / gcd(a, b) * b; }

// Returns GCD and coefficients of Bézout's identity (ax + by = gcd(a,b))
tuple<long long, long long, long long> extended_gcd(long long a, long long b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x1, y1] = extended_gcd(b, a % b);
    return {g, y1, x1 - (a / b) * y1};
}

// If we need to find the modular multiplicative inverse of 'a' modulo 'm'
// (i.e., find x such that ax ≡ 1 (mod m))
long long modInverse(long long a, long long m) {
    auto [g, x, y] = extended_gcd(a, m);
    if (g != 1) return -1; // Inverse doesn't exist
    return (x % m + m) % m; // Make sure the answer is positive
}

// Solves equations of the form: ax + by = c
bool solveLinearDiophantine(long long a, long long b, long long c,
                             long long &x, long long &y) {
    auto [g, x0, y0] = extended_gcd(abs(a), abs(b));
    if (c % g != 0) return false; // No solution exists
    x = x0 * (c / g);
    y = y0 * (c / g);
    if (a < 0) x = -x;
    if (b < 0) y = -y;
    return true;
}

```

Modular Arithmetic

```

const int MOD = 1e9 + 7;

long long mod_add(long long a, long long b) { return ((a % MOD) + (b % MOD)) % MOD; }
long long mod_mul(long long a, long long b) { return ((a % MOD) * (b % MOD)) % MOD; }

long long mod_pow(long long base, long long exp) { // Modular Exponentiation (Fast Power)
    long long result = 1; base %= MOD;
    while (exp > 0) {
        if (exp & 1) result = mod_mul(result, base);
        base = mod_mul(base, base); exp >>= 1;
    }
    return result;
}

// a^e mod M
long long modpow(long long a, long long e, long long M) {
    long long r = 1 % M;
    while (e) {
        if (e & 1) r = (__int128)r * a % M; // __int128 := 128-bit integer
        a = (__int128)a * a % M;
        e >>= 1;
    }
    return r;
}

```



```
// Modular Multiplicative Inverse (Only works when MOD is prime)
long long mod_inverse(long long a) { return mod_pow(a, MOD - 2); }

// Modular inverse using Extended Euclidean Algorithm
// Works for any coprime a and mod
long long mod_inverse_extended(long long a, long long mod) {
    auto [g, x, y] = extended_gcd(a, mod);
    if (g != 1) { // Inverse doesn't exist if a and mod are not coprime
        throw invalid_argument("Inverse does not exist: a and mod must be coprime");
    }
    return (x % mod + mod) % mod; // Ensure positive result
}

// Used as part of solving systems of linear congruences
// For example: solving x ≡ a1 (mod m1), x ≡ a2 (mod m2), ...
long long chinese_remainder(vector<long long>& num, vector<long long>& rem) {
    long long prod = 1, result = 0;
    for (long long x : num) prod *= x;
    for (int i = 0; i < num.size(); i++) {
        long long pp = prod / num[i];
        result = mod_add(result,
            mod_mul(mod_mul(rem[i], pp), mod_inverse_extended(pp, num[i])));
    }
    return result % prod;
}
```

Factorial

```
const int MAX_N = 1000005;
vector<long long> factorial(MAX_N), inv_factorial(MAX_N);

void precomputeFactorials() {
    factorial[0] = 1;
    for (int i = 1; i < MAX_N; i++) { factorial[i] = mod_mul(factorial[i-1], i); }
    inv_factorial[MAX_N-1] = mod_pow(factorial[MAX_N-1], MOD-2);
    for (int i = MAX_N-2; i >= 0; i--) inv_factorial[i] = mod_mul(inv_factorial[i+1], i+1);
}

long long nCr(int n, int r) {
    if (r > n) return 0;
    return mod_mul(factorial[n], mod_mul(inv_factorial[r], inv_factorial[n-r]));
}
```

Prime

```
bool isPrime(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 3; i * i <= n; i += 2) if (n % i == 0) return false;
    return true;
}

// Sieve of Eratosthenes
vector<bool> isPrime(MAX, true);
vector<int> primes;
void sieve() {
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= MAX; i++)
        if (isPrime[i]) for (int j = i * i; j <= MAX; j += i) isPrime[j] = false;
    for (int i = 2; i <= MAX; i++) if (isPrime[i]) primes.push_back(i);
}

// Segmented Sieve - For finding primes in a range [L, R]
vector<bool> segmentedSieve(long long L, long long R) {
    long long lim = sqrt(R);
    vector<bool> mark(lim + 1, false);
    vector<long long> primes;
    for (long long i = 2; i <= lim; ++i)
```

```

        if (!mark[i]) {
            primes.push_back(i);
            for (long long j = i * i; j <= lim; j += i) mark[j] = true;
        }
        vector<bool> isPrime(R - L + 1, true);
        for (long long i : primes)
            for (long long j = max(i * i, (L + i - 1) / i * i); j <= R; j += i)
                isPrime[j - L] = false;
        if (L == 1) isPrime[0] = false;
        return isPrime;
    }

    vector<pair<long long, int>> primeFactorize(long long n) {
        vector<pair<long long, int>> factors;
        for (long long i = 2; i * i <= n; i++)
            if (n % i == 0) {
                int count = 0;
                while(n % i == 0) { cnt++; n /= i; }
                factors.push_back({i, count});
            }
        if (n > 1) factors.push_back({n, 1});
        return factors;
    }

    // counts how many integers in the range 1 <= k <= n are coprime to n
    // if p is prime then phi(p) = p - 1
    // if p is prime and and k >= 1 then phi(p^k) = p^k - p^(k-1)
    // phi(ab) = phi(a)phi(b) iff gcd(a, b) = 1
    int phi(int n) {
        int result = n;
        for (int i = 2; i * i <= n; i++)
            if (n % i == 0) { while(n % i == 0) n /= i; result -= result / i; }
        if (n > 1) result -= result / n;
        return result;
    }

    vector<int> precomputePhi(int N) {
        vector<int> phi(N+1), primes; vector<bool> comp(N+1, false);
        phi[0] = 0, phi[1] = 1;
        for(int i = 2; i <= N; i++) {
            if (!comp[i]) primes.push_back(i); phi[i] = i - 1;
            for (int p : primes) {
                long long v = 1LL * p * i;
                if (v > N) break;
                comp[v] = true;
                if (i % p == 0) { phi[v] = phi[i] * p; break; }
                else phi[v] = phi[i] * (p - 1);
            }
        }
        return phi;
    }
}

```

Divisor Count and Sum

```

struct DivInfo { int count; long long sum; };

vector<DivInfo> divisors_count_sum(int n) {
    vector<DivInfo> info(n + 1, {0, 0});
    for(int i = 1; i <= n; i++) for(int j = i; j <= n; j += i)
        info[j].count++; info[j].sum += i;
    return info;
}

```

Other Maths

Matrices

```
struct Mat {
    int n;
    vector<vector<long long>>> m;
    long long MOD;

    Mat(int n, long long MOD, bool ident = false) :
        n(n), m(n, vector<long long>(n, 0)), MOD(MOD){
        if (ident) for(int i = 0; i < n; i++) m[i][i] = 1;
    }

    Mat operator*(Mat& o) const {
        Mat r(n, MOD);
        for (int i = 0; i < n; i++) for (int k = 0; k < n; k++) if (m[i][k])
            for (int j = 0; j < n; j++) r.m[i][j] = (r.m[i][j] + m[i][k] * o.m[k][j]) % MOD;
        return r;
    }

    Mat operator+(Mat& o) const {
        if (n != o.n || MOD != o.MOD) throw invalid_argument("Matrix dimension/mod mismatch");
        Mat r(n, MOD);
        for (int i = 0; i < n; i++) for (int j = 0; j < n; j++)
            r.m[i][j] = (m[i][j] + o.m[i][j]) % MOD;
        return res;
    }

    Mat operator-(Mat& o) const {
        if (n != o.n || MOD != o.MOD) throw invalid_argument("Matrix dimension/mod mismatch");
        Mat r(n, MOD);
        for (int i = 0; i < n; i++) for (int j = 0; j < n; j++)
            r.m[i][j] = (m[i][j] - o.m[i][j] + MOD) % MOD;
        return r;
    }

    Mat operator/(long long scalar) const { // Scalar Division
        auto mod_inv = [&](long long a) {
            long long m = MOD, y = 0, x = 1;
            while(a > 1) {
                long long q = a / m;
                tie(a, m) = make_tuple(m, a % m); tie(x, y) = make_tuple(y, x - q * y);
            }
            return x < 0 ? x + MOD : x;
        };
        long long inv = mod_inv(scalar);
        Mat res(n, MOD);
        for (int i = 0; i < n; i++) for (int j = 0; j < n; j++)
            res.m[i][j] = (m[i][j] * inv) % MOD;
        return res;
    }

    Mat inverse() const {
        if(n != m[0].size()) throw invalid_argument("Matrix must be square for inversion");
        vector<vector<long long>>> aug(m);
        int mod = MOD;

        for (int i = 0; i < n; i++) { aug[i].resize(2*n, 0); aug[i][i+n] = 1; }
        auto mod_inv = [&](long long a) {
            long long m = mod, y = 0, x = 1;
            while (a > 1) {
                long long q = a/m;
                tie(a, m) = make_tuple(m, a%m); tie(x, y) = make_tuple(y, x - q*y);
            }
            return x < 0 ? x + mod : x;
        };

        for (int col = 0; col < n; col++) {
            int pivot = -1;
            for (int i = col; i < n; i++)
                if (aug[i][col] != 0 && gcd(aug[i][col], mod) == 1) { pivot = i; break; }
            if (pivot == -1)
                throw runtime_error("Matrix is singular modulo " + to_string(mod));
            swap(aug[col], aug[pivot]);
            long long inv = mod_inv(aug[col][col]);
```

```

        for (int j = col; j < 2*n; j++) {
            aug[col][j] = (aug[col][j] * inv) % mod;
            if(aug[col][j] < 0) aug[col][j] += mod;
        }
        for (int i = 0; i < n; i++)
            if (i != col && aug[i][col] != 0) {
                long long factor = aug[i][col];
                for (int j = col; j < 2*n; j++) {
                    aug[i][j] = (aug[i][j] - factor * aug[col][j]) % mod;
                    if(aug[i][j] < 0) aug[i][j] += mod;
                }
            }
    }

    Mat inv(n, mod);
    for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) inv.m[i][j] = aug[i][j+n];
    return inv;
}

};

Mat matPow(Mat base, long long exp){
    Mat res(base.n, base.MOD, true);
    while (exp) {
        if (exp&1) res = res * base;
        base = base * base; exp >>= 1;
    }
    return res;
}

long long fib_mod(long long n, long long M){
    if (n == 0) return 0;
    Mat T(2,M); T.m = {{1,1}, {1,0}}; Mat P = matPow(T,n-1);
    return P.m[0][0] % M;    // F_n
}

```

Arithmetic / Geometric Progressions

```

int S = n * (a_1 + a_n) / 2;
int S_n = n * (n + 1) / 2;
int S_n_square = n * (n + 1) * (2*n + 1) / 6;
int S_n_cube = (n * (n + 1) / 2) ** 2;

```

Geometry

```

const double EPS = 1e-9;
int sgn(double x){ return fabs(x) < EPS ? 0 : (x < 0 ? -1 : 1); }

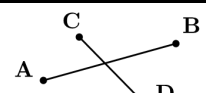
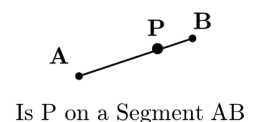
struct Point {
    double x, y;
    Point() {}
    Point(double x, double y): x(x), y(y) {}
    Point operator+(Point& o) const { return {x + o.x, y + o.y}; }
    Point operator-(Point& o) const { return {x - o.x, y - o.y}; }
    Point operator*(double k) const { return {x * k, y * k}; }
};

double dot(Point a, Point b){ return a.x * b.x + a.y * b.y; }
double cross(Point a, Point b){ return a.x * b.y - a.y * b.x; }
double cross(Point a, Point b, Point c){ return cross(b - a, c - a); }    // 0: Collinear
double norm2(Point a){ return dot(a,a); }
double dist(Point a, Point b){ return sqrt(norm2(a - b)); }

bool onSegment(Point a, Point b, Point p) {
    if(sgn(cross(a, b, p)) != 0) return false;
    return (min(a.x, b.x) - EPS <= p.x && p.x <= max(a.x,b.x) + EPS &&
            min(a.y, b.y) - EPS <= p.y && p.y <= max(a.y,b.y) + EPS);
}

bool segIntersect(Point a, Point b, Point c, Point d){
    double c1 = cross(a, b, c), c2 = cross(a, b, d), c3 = cross(c, d, a), c4 = cross(c, d, b);

```



```

if(sgn(c1) * sgn(c2) < 0 && sgn(c3) * sgn(c4) < 0) return true;
if(sgn(c1) == 0 && onSegment(a, b, c)) return true;
if(sgn(c2) == 0 && onSegment(a, b, d)) return true;
if(sgn(c3) == 0 && onSegment(c, d, a)) return true;
if(sgn(c4) == 0 && onSegment(c, d, b)) return true;
return false;
}
// Treats Point as Vector
Point lineIntersection(Point p, Point r, Point q, Point s) {
    double t = cross(q - p, s) / cross(r, s);
    return p + r * t;
}

double polygonArea(vector<Point>& P) {
    double s = 0; int n = P.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        s += P[i].x * P[j].y - P[j].x * P[i].y;
    }
    return fabs(s) / 2;
}

// Convex Hull (Monotonic chain). Returns CCW hull without duplicate last point.
vector<Point> convexHull(vector<Point> pts){
    sort(pts.begin(), pts.end(), [](Point &a, Point &b) {
        if (sgn(a.x - b.x) == 0) return a.y < b.y;
        return a.x < b.x;
    });
    vector<Point> lo, up;
    for (auto &p : pts) {
        while (lo.size() >= 2 && cross(lo[lo.size() - 2], lo.back(), p) <= 0) lo.pop_back();
        lo.push_back(p);
    }
    for (int i = (int)pts.size() - 1; i >= 0; i--) {
        auto p = pts[i];
        while (up.size() >= 2 && cross(up[up.size() - 2], up.back(), p) <= 0) up.pop_back();
        up.push_back(p);
    }
    lo.pop_back(); up.pop_back();
    lo.insert(lo.end(), up.begin(), up.end());
    return lo;
}

struct Circle { Point c; double r; };

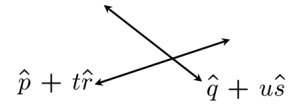
// Constructs a line from a, b and returns the number of intersection point(s)
int circleLineIntersect(Point a, Point b, Circle C, vector<Point>& out) {
    Point d = b - a, f = a - C.c;
    double A = dot(d, d), B = 2 * dot(f, d), Cc = dot(f, f) - C.r * C.r,
           disc = B * B - 4 * A * Cc;
    if (disc <= EPS) return 0;
    if (fabs(disc) < EPS) { double t = -B / (2 * A); out.push_back(a + d * t); return 1; }
    disc = sqrt(disc);
    double t1 = (-B - disc) / (2 * A), t2 = (-B + disc) / (2 * A);
    out.push_back(a + d * t1); out.push_back(a + d * t2);
    return 2;
}

double triangleArea(Point a, Point b, Point c) { return fabs(cross(a, b, c)) / 2.0; }

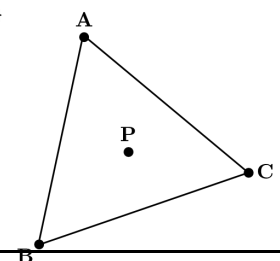
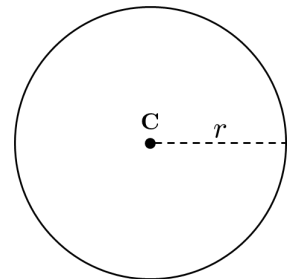
bool isTriangle(Point a, Point b, Point c) { return cross(a, b, c) != 0; }

double triangleAreaHeron(double s1, double s2, double s3) {
    if (s1 + s2 <= s3 + EPS || s1 + s3 <= s2 + EPS ||
        s2 + s3 <= s1 + EPS) return 0;
    double s = (s1 + s2 + s3) / 2.0;
    return sqrt(s * (s - s1) * (s - s2) * (s - s3));
}

```



$\hat{p} + t\hat{r}$
 $\hat{q} + u\hat{s}$
 A Line $\hat{p} + t\hat{r}$ intersects
 a Line $\hat{q} + u\hat{s}$



Is P inside a Triangle ABC

```

bool isInsideTriangle(Point a, Point b, Point c, Point p) {
    double s1 = cross(a, b, p), s2 = cross(b, c, p), s3 = cross(c, a, p);
    if (sgn(s1) >= 0 && sgn(s2) >= 0 && sgn(s3) >= 0) return true;
    if (sgn(s1) <= 0 && sgn(s2) <= 0 && sgn(s3) <= 0) return true;
    return false;
}

Point circumcenter(Point a, Point b, Point c) {
    Point ab_mid = (a + b) * 0.5;
    ab_perp_dir = b - a;
    swap(ab_perp_dir.x, ab_perp_dir.y);
    ab_perp_dir.x *= -1;
    Point bc_mid = (b + c) * 0.5;
    bc_perp_dir = c - b;
    swap(bc_perp_dir.x, bc_perp_dir.y);
    bc_perp_dir.x *= -1;
    return lineIntersection(ab_mid, ab_perp_dir, bc_mid, bc_perp_dir);
}

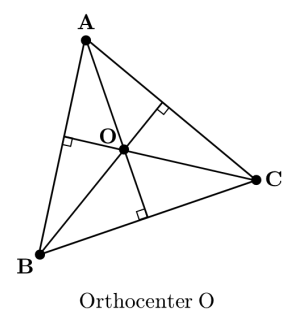
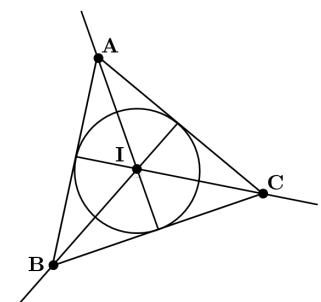
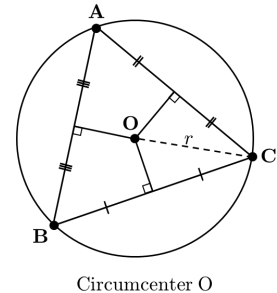
double circumradius(Point a, Point b, Point c) {
    double s_a = dist(b, c), s_b = dist(a, c), s_c = dist(a, b),
        area = triangleArea(a, b, c);
    if (sgn(area) == 0) return INFINITY; // Degenerate triangle
    return (s_a * s_b * s_c) / (4.0 * area);
}

Point incenter(Point a, Point b, Point c) {
    double s_a = dist(b, c), s_b = dist(a, c), s_c = dist(a, b);
    return (a * s_a + b * s_b + c * s_c) * (1.0 / (s_a + s_b + s_c));
}

Point orthocenter(Point a, Point b, Point c) {
    Point ab_perp_dir = b - a;
    swap(ab_perp_dir.x, ab_perp_dir.y); ab_perp_dir.x *= -1;
    Point ac_perp_dir = c - a;
    swap(ac_perp_dir.x, ac_perp_dir.y); ac_perp_dir.x *= -1;
    return lineIntersection(c, ab_perp_dir, b, ac_perp_dir);
}

Circle smallestEnclosingCircle(Point a, Point b, Point c) {
    double a2 = norm2(b - c), b2 = norm2(a - c), c2 = norm2(a - b);
    if (a2 >= b2 + c2) return { (b + c) * 0.5, sqrt(a2) * 0.5 };
    else if (b2 >= a2 + c2) return { (a + c) * 0.5, sqrt(b2) * 0.5 };
    else if (c2 >= a2 + b2) return { (a + b) * 0.5, sqrt(c2) * 0.5 };
    return { circumcenter(a, b, c), dist(center, a) };
}

```



Trigonometry

Law of cosines

$$\begin{aligned}
 a^2 &= b^2 + c^2 - 2bc \cos \alpha \\
 b^2 &= a^2 + c^2 - 2ac \cos \beta \\
 c^2 &= a^2 + b^2 - 2ab \cos \gamma
 \end{aligned}$$

Law of sines

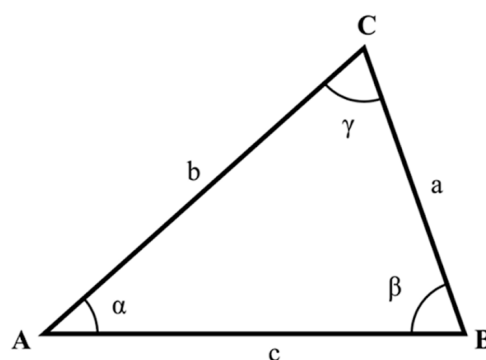
$$\frac{a}{\sin \alpha} = \frac{b}{\sin \beta} = \frac{c}{\sin \gamma} = 2R$$

Sum of angles

$$\alpha + \beta + \gamma = 180$$

Law of tangents

$$\frac{a-b}{a+b} = \frac{\tan \left[\frac{1}{2}(\alpha - \beta) \right]}{\tan \left[\frac{1}{2}(\alpha + \beta) \right]}$$



Plane triangle with side lengths a, b, c, and three angular α , β , γ

$$\sin^2(\theta) + \cos^2(\theta) = 1$$

$$1 + \tan^2(\theta) = \sec^2(\theta)$$

$$1 + \cot^2(\theta) = \csc^2(\theta)$$

$$\sin(A \pm B) = \sin(A) \cos(B) \pm \cos(A) \sin(B)$$

$$\cos(A \pm B) = \cos(A) \cos(B) \mp \sin(A) \sin(B)$$

$$\tan(A \pm B) = \frac{\tan(A) \pm \tan(B)}{1 \mp \tan(A) \tan(B)}$$

$$\sin(2\theta) = 2 \sin(\theta) \cos(\theta)$$

$$\cos(2\theta) = \cos^2(\theta) - \sin^2(\theta)$$

$$\cos(2\theta) = 2 \cos^2(\theta) - 1$$

$$\cos(2\theta) = 1 - 2 \sin^2(\theta)$$

$$\tan(2\theta) = \frac{2 \tan(\theta)}{1 - \tan^2(\theta)}$$

$$A = \frac{1}{2} ab \sin(C)$$

Geometric Transformations with Matrix

2D Translation

2D Scaling

2D Rotation

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \quad \begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

3D Translation

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

3D Scaling

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

3D Rotation about the x-axis

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

3D Rotation about the y-axis

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

3D Rotation about the z-axis

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Range Query

```
struct MoAlgorithm {
    struct Query { int l, r, idx, block_idx; };
    vector<int> arr, freq;
    int current_distinct = 0, block_size;
    void add(int idx) { if (freq[arr[idx]] == 0) current_distinct++; freq[arr[idx]]++; }
    void remove(int idx) { if (freq[arr[idx]] == 1) current_distinct--; freq[arr[idx]]--; }
    bool compareQueries(Query &a, Query &b) {
        if (a.block_idx != b.block_idx) return a.block_idx < b.block_idx;
        return (a.block_idx & 1) ? (a.r < b.r) : (a.r > b.r);
    }
}

struct SqrtDecomposition {
    vector<long long> arr, blocks;
    int block_size, n;

    SqrtDecomposition(vector<long long>& initial_arr) {
        arr = initial_arr; n = arr.size();
        block_size = static_cast<int>(sqrt(n));
        if (block_size == 0) block_size = 1;
        blocks.assign(ceil((double)n / block_size), 0);
        for (int i = 0; i < n; ++i) blocks[i / block_size] += arr[i];
    }

    void update(int idx, long long val) {
        if (idx < 0 || idx >= n) return;
        int block_idx = idx / block_size; blocks[block_idx] += val - arr[idx];
        arr[idx] = val;
    }

    long long query_sum(int l, int r) {
        if (l > r || l < 0 || r >= n) return 0;
        long long sum = 0;
        int block_l = l / block_size, block_r = r / block_size;
        if (block_l == block_r)
            for (int i = l; i <= r; ++i) sum += arr[i];
        else {
            for (int i = l; i < (block_l + 1) * block_size; ++i) sum += arr[i];
            for (int i = block_l + 1; i < block_r; ++i) sum += blocks[i];
            for (int i = block_r * block_size; i <= r; ++i) sum += arr[i];
        }
        return sum;
    }
};

struct SegmentTree {
    vector<long long> tree, arr;
    int n;
```

```

void build(int node, int start, int end) {
    if (start == end) { tree[node] = arr[start]; return; }
    int mid = (start + end) / 2;
    build(2 * node, start, mid); build(2 * node + 1, mid + 1, end);
    tree[node] = tree[2 * node] + tree[2 * node + 1];
}

void update_tree(int node, int start, int end, int idx, long long val) {
    if (start == end) { arr[idx] = val; tree[node] = val; return; }
    int mid = (start + end) / 2;
    if (start <= idx && idx <= mid) update_tree(2 * node, start, mid, idx, val);
    else update_tree(2 * node + 1, mid + 1, end, idx, val);
    tree[node] = tree[2 * node] + tree[2 * node + 1];
}

long long query_tree(int node, int start, int end, int l, int r) {
    if (r < start || end < l) return 0;
    if (l <= start && end <= r) return tree[node];
    int mid = (start + end) / 2;
    long long p1 = query_tree(2 * node, start, mid, l, r),
              p2 = query_tree(2 * node + 1, mid + 1, end, l, r);
    return p1 + p2;
}

SegmentTree(vector<long long>& initial_arr) {
    this->arr = initial_arr;
    n = arr.size();
    tree.assign(4 * n, 0); // Tree size is roughly 4*N
    build(1, 0, n - 1);
}

void update(int idx, long long val) { update_tree(1, 0, n - 1, idx, val); }

long long query(int l, int r) { return query_tree(1, 0, n - 1, l, r); }
};

```

Random

```

struct Random {
    mt19937 rng;

    Random() : rng(chrono::steady_clock::now().time_since_epoch().count()) {}
    Random(int seed) : rng(seed) {}

    // Random integer in range [a, b]
    int randint(int a, int b) { return uniform_int_distribution<int>(a, b)(rng); }

    // Random double in range [a, b]
    double randdouble(double a, double b) {
        return uniform_real_distribution<double>(a, b)(rng);
    }

    // Random boolean with probability p
    bool randbool(double p = 0.5) { return bernoulli_distribution(p)(rng); }

    // Shuffle container
    template<typename T>
    void shuffle(T& container) { std::shuffle(container.begin(), container.end(), rng); }

    // Random choice from vector
    template<typename T>
    T choice(const vector<T>& vec) { return vec[randint(0, vec.size() - 1)]; }
};

int main() {
    Random rnd;

    cout << "Random int [1, 100]: " << rnd.randint(1, 100) << endl;
    cout << "Random double [0, 1): " << rnd.randdouble(0.0, 1.0) << endl;
}

```



```
cout << "Random bool: " << rnd.randbool() << endl;

vector<int> arr = {1, 2, 3, 4, 5};
rnd.shuffle(arr);
cout << "Shuffled array: ";
for (int x : arr) cout << x << " ";
cout << endl;
cout << "Random choice: " << rnd.choice(arr) << endl;
return 0;
}
```

Statistics

```
vector<int> v(n);

double mean = accumulate(v.begin(), v.end(), 0) / (double) n;
sort(v.begin(), v.end());
int median = n % 2 == 0 ? (v[n / 2 - 1] + v[n / 2]) / 2.0 : v[n / 2];

map<int, int> freq;
int max_freq = 0, mode = v[0];
for (int val : v) {
    freq[val]++;
    if (freq[val] > max_freq) {
        max_freq = freq[val]; mode = val;
    }
}

double variance = accumulate(v.begin(), v.end(), 0.0, [mean](double acc, int x) {
    return acc + pow(x - mean, 2);
}) / n;

double standard_deviation = sqrt(variance);

// Linear Regression
// Calculates the slope (m) and y-intercept (c) of a line
// that best fits a set of points using the method of least squares.
struct LinearRegression {
    vector<pair<double, double>> points;
    double m, c; // y = mx + c

    LinearRegression(vector<pair<double, double>> points) : points(points) { }

    void calculate() {
        if (points.size() < 2) { m = 0.0; c = 0.0; return; }
        double n = static_cast<double>(points.size());
        double sum_x = 0.0, sum_y = 0.0, sum_xy = 0.0, sum_x_sq = 0.0;
        for (auto p : points) {
            sum_x += p.first; sum_y += p.second;
            sum_xy += p.first * p.second;
            sum_x_sq += p.first * p.first;
        }
        m = (n * sum_xy - sum_x * sum_y) / (n * sum_x_sq - sum_x * sum_x);
        c = (sum_y - m * sum_x) / n;
    }
}

// Normal Distribution
// Calculates the probability density function (PDF) of a normal distribution.
double normal_pdf(double x, double mean, double std_dev) {
    double exponent = - pow(x - mean, 2) / (2 * pow(std_dev, 2));
    return (1.0 / (std_dev * sqrt(2.0 * M_PI))) * exp(exponent);
}

// Calculates the cumulative distribution function (CDF) of a normal distribution.
// This is the probability that a random variable X will be less than or equal to x.
// Uses the error function (erf) from <cmath>.
double normal_cdf(double x, double mean, double std_dev) {
    double z = (x - mean) / (std_dev * sqrt(2.0));
    return 0.5 * (1.0 + erf(z));
}
```